Lab 2 — Data Cleaning and Exploratory Data Analysis (EDA)

Student version

Institution	ISIMM — Higher Institute of Computer Science and Mathematics of Monastir
Programme / level	3rd-year engineering — Software Engineering
Course	Machine Learning — Chapter 2: Data Preparation and EDA
Duration	90 minutes
Format	Pairs — formative assessment (not graded)
Dataset	Titanic (891 passengers, binary target `survived`)

1. Objectives

By the end of the session you will be able to:

- **O1** — Audit a dataset: dimensions, types, missing values.
- **O2** — Apply the anti-leakage rule: **split train/test before computing any statistic**.
- **O3** — Carry out an EDA on the training set and interpret the results.
- **O4** — Build a preprocessing pipeline with `ColumnTransformer`: imputation, encoding, standardization.
- **O5** — Check preprocessing with assertions and diagnose data leakage in a given piece of code.

2. Prerequisites

- Lab 1: working with NumPy arrays and Pandas DataFrames (`groupby`, `isna`, indexing).
- Chapter 2: missing values, encoding, scaling, correlation, **data leakage**.

3. Scenario

A team hands you a historical dataset to prepare for a future classification model. Before any modelling, you must deliver data that is **clean, documented and prepared without information leakage**. Badly prepared data produces over-optimistic evaluation scores, and this often goes unnoticed. Your deliverable: a notebook that audits, explores and prepares the data, with automatic checks.

4. Environment setup (Exercise 0 — 10 min)

Files provided in the lab folder:

- `Lab02_starter.ipynb`: the notebook to complete.
- `data/titanic.csv`: a local copy of the dataset, for working offline.

Steps

1. Open a terminal **in the lab folder**, the one containing `Lab02_starter.ipynb` and `data/`.
2. Check that Python 3.10 or later is installed: `python --version`.
3. Install the dependencies if they are missing:

```
pip install numpy pandas matplotlib seaborn scikit-learn notebook
```

4. Start Jupyter with `jupyter notebook`, then open `Lab02_starter.ipynb`.
5. Run the first cell. It must print `(891, 8)`.

Fallback. Without Internet, the code automatically loads `data/titanic.csv` through `sns.load_dataset("titanic", data_home="data")`. If installation fails, tell the instructor.

Prerequisite warm-up. Answer in writing, in two minutes at most:

- a. What does `df.isna().sum()` return?
- b. What is the difference between `fit` and `transform` for a scikit-learn transformer?
- c. Define data leakage in one sentence.

5. Data

Variable	Type	Description
`survived`	integer 0/1	**Target**: 1 = survived
`pclass`	integer 1/2/3	Ticket class (1 = first)
`sex`	text	`male` / `female`
`age`	float (years)	Passenger age
`sibsp`	integer	Number of siblings and spouses aboard
`parch`	integer	Number of parents and children aboard
`fare`	float	Ticket price (historical currency, not converted)
`embarked`	text	Port of embarkation: `C`, `Q`, `S`

Source. The `titanic` dataset distributed with the seaborn library. It is real historical data, not synthetic data.

Limitations. This data reflects the social rules of 1912 (priority to women and to the wealthier classes). It is used for learning. It does not model any present-day situation.

6. Exercises

> **Golden rule of this lab:** no statistic (median, mean, mode, standard deviation, categories) may be computed on data that contains the test set.

Exercise 1 — Dataset audit (10 min) · O1

Prediction (before running). Which columns do you think contain missing values? In what proportion?

1. Display `df.shape`, `df.dtypes` and `df.head()`.
2. Compute `missing = df.isna().sum()` and print it.
3. Compute the percentage of missing values in `age`.
4. Compute the overall survival rate.
5. Plot a bar chart of `missing`.

Deliverable: executed cell, then a written answer: *which columns will need imputation?*

Checkpoint: you should find two incomplete columns. Compare with your prediction.

Graded hints

1. `isna()` returns a table of booleans. What does their sum give?
2. Percentage = number missing / `len(df)` × 100.
3. Survival rate = mean of a 0/1 variable.

Exercise 2 — Split first (5 min) · O2

1. Complete `train_test_split` with `test_size=0.25`, `random_state=42` and `stratify=y`.
2. Print the shapes, then the survival rate of `y_tr` and of `y_te`.
3. Add an assertion checking that the train and test indices are disjoint.

Question 2.4. What is `stratify=y` for? Compare the two survival rates you obtain.

Checkpoint: train = 668 rows, test = 223 rows.

Graded hints

1. The function returns four objects, in the order `X_tr, X_te, y_tr, y_te`.
2. `set(X_tr.index).isdisjoint(X_te.index)`.

Exercise 3 — Exploratory analysis on the training set (15 min) · O3

Work **only** on `train = X_tr.assign(survived=y_tr)`.

1. Compute the survival rate by `sex` and by `pclass`, then plot two bar charts.
2. Compute the mean and the median of `fare` and of `age`, and the skewness of `fare` with `.skew()`.
3. Compute the correlation matrix of `age`, `fare`, `sibsp`, `parch` and `survived`, then display it as a heatmap with `sns.heatmap`.
4. Plot the histograms of `age` and of `fare`.

Interpretation questions

- **3.5** Which variables seem most related to survival? Justify with your numbers.
- **3.6** Compare the mean and the median of `fare`. What does the gap reveal? If `fare` had missing values, which imputation value would you choose, and why?
- **3.7** Why do we carry out the EDA on the training set rather than on the full dataset?

Graded hints

1. `train.groupby("sex")["survived"].mean()`.
2. A skewness well above 1 indicates a long right tail.
3. For 3.7, re-read the golden rule of this lab.

Exercise 4 — Leakage-safe preprocessing pipeline (20 min) · O4

Prediction. After transformation, how many columns will the matrix have? Show your calculation: 4 numeric columns, plus how many columns for each categorical variable?

1. `numeric_pipe`: `SimpleImputer(strategy="median")` then `StandardScaler()`.
2. `categorical_pipe`: `SimpleImputer(strategy="most_frequent")` then `OneHotEncoder(handle_unknown="ignore")`.
3. `pre`: a `ColumnTransformer` that routes `numeric` to the first pipeline and `categorical` (`sex`, `embarked`, `pclass`) to the second, with `sparse_threshold=0`.
4. Learn the statistics on the training set: `X_tr_prep = pre.fit_transform(X_tr)`.

5. Apply them to the test set: `X_te_prep = pre.transform(X_te)`.
6. Print the shapes and `pre.get_feature_names_out()`.

Questions

- **4.7** Why one-hot encode `sex` and `embarked` rather than use integer codes (0, 1, 2)?
- **4.8** Here, `pclass` is also one-hot encoded. Give one argument for this choice and one argument for keeping `pclass` as it is (ordinal variable).
- **4.9** What does `handle_unknown="ignore"` protect against?

Checkpoint: the training and test matrices have the same number of columns. Does this number match your prediction?

Graded hints

1. `Pipeline([("imp", ...), ("sc", ...)])`.
2. `ColumnTransformer([("num", numeric_pipe, numeric), ("cat", categorical_pipe, categorical)], sparse_threshold=0)`.
3. You **never** call `fit` on `X_te`.

Exercise 5 — Check and diagnose (10 min) · O5

5a — Automatic checks

1. Add an assertion checking that no `NaN` remains (`np.isnan(...).any()`).
2. Add an assertion checking that the first four columns of `X_tr_prep` have a mean of about 0 and a standard deviation of about 1 (`np.allclose`).
3. Print the means and standard deviations of the four numeric columns of `X_te_prep`.

Prediction 5.3. Will the test means be exactly 0? Justify **before** running.

5b — Find the bug. The notebook contains the following code:

```
scaler = StandardScaler()
fare_scaled = scaler.fit_transform(X[["fare"]])
fs_tr, fs_te = train_test_split(fare_scaled, test_size=0.25,
                                random_state=42, stratify=y)
```

- **5.4** This code runs without error. Where is the problem? Answer **before** running it.
- **5.5** Write the corrected version. Compare the value of `scaler.mean_` in both versions.

Graded hints

1. On which rows is the mean of `fare` computed in this code?
2. Do those rows include test passengers?

7. Optional extensions (outside the required time)

- **E1 — Log transform.** Apply `np.log1p` to `fare`, then compare the skewness and the histogram before and after.
- **E2 — Missingness indicator.** Compare the survival rate of passengers whose age is missing with that of the others. Then try `SimpleImputer(strategy="median", add_indicator=True)`. How many columns do you get?
- **E3 — Imputation artefact.** How many ages are imputed in the training set? Plot the histogram of `age` after imputation and comment.

8. Summary to complete

1. The correct order of steps is: audit → _____ → EDA (on _____) → pipeline `fit` on _____ → `transform` on _____.
2. The median is preferable to the mean for imputing a _____ variable.
3. Standardization centres and scales, but does not remove _____ .
4. Data leakage makes the evaluation _____ .

9. Individual knowledge check (5 min, without the notebook)

1. Which of the two methods learns statistics: `fit` or `transform`?
2. Explain the number of columns obtained after preprocessing.
3. A variable standardized on the training set has a mean of 0.09 on the test set. Is this a bug? Justify.
4. Does standardization fix the skewness of `fare`?
5. Is the large difference in survival by `sex` a modelling opportunity, a fairness issue, or both? Argue in three lines.

10. Submission and success criteria

Submit at the end of the session: the notebook `Lab02_<name1>_<name2>.ipynb`, run top to bottom without error, with your written answers in the Markdown cells.

Criterion	Achieved	In progress	Not achieved
Complete audit: missing values identified and quantified (O1)			
Split done before any statistic, with assertion (O2)			
EDA on the training set, interpretation backed by numbers (O3)			
Correct `ColumnTransformer`, `fit` on train only (O4)			
Assertions pass and leakage bug explained (O5)			
Notebook re-runs top to bottom			

French – English glossary

Français	English
valeur manquante	missing value
imputation	imputation
fuite de données	data leakage
ensemble d’apprentissage / de test	training / test set
séparation stratifiée	stratified split
encodage one-hot	one-hot encoding
standardisation	standardization
asymétrie	skewness
analyse exploratoire	exploratory data analysis (EDA)
catégorie inconnue	unknown category